Erklären Sie mit eigenen Worten, was Softwarequalität bedeutet.

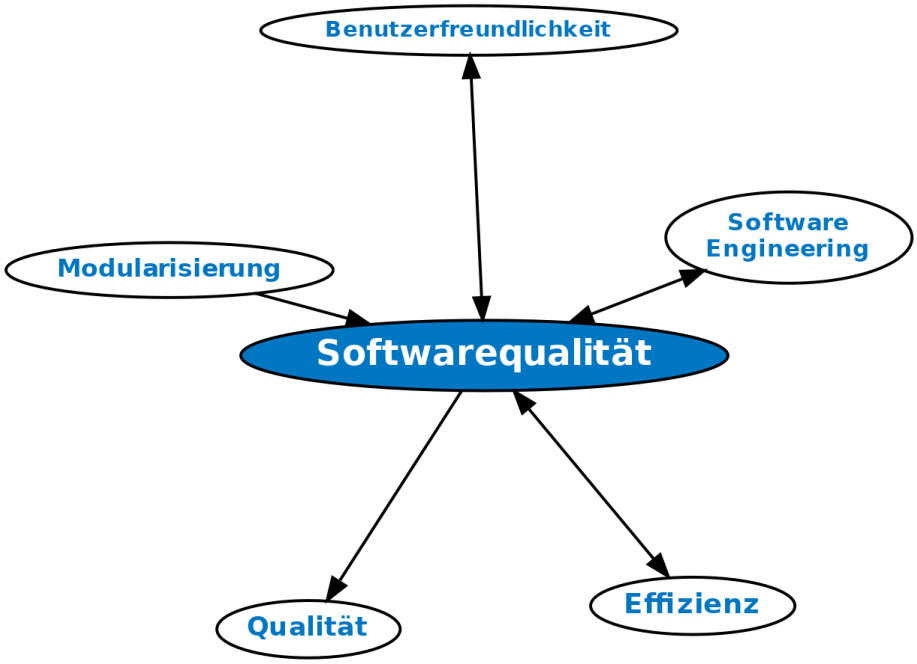
Softwarequalität beschreibt, wie gut eine Software funktioniert und ob sie die Bedürfnisse der Nutzer erfüllt.

Wichtige Punkte sind:

1. Funktionalität: Die Software sollte die gewünschten Aufgaben korrekt ausführen.
2. Zuverlässigkeit: Sie sollte stabil laufen und möglichst wenig Fehler aufweisen.
3. Benutzerfreundlichkeit: Die Anwendung sollte leicht zu bedienen sein und eine gute Nutzererfahrung bieten.
4. Effizienz: Die Software sollte Ressourcen optimal nutzen und schnell reagieren.
5. Wartbarkeit: Sie sollte einfach zu aktualisieren und anzupassen sein.

Insgesamt geht es darum, dass die Software nicht nur technisch gut ist, sondern auch den Erwartungen der Nutzer gerecht wird. Hochwertige Software sorgt für Zufriedenheit und minimiert Probleme im Gebrauch.

Erstellen Sie eine Mindmap von Softwarequalität und stellen Sie deren Aspekte dar.



Erklären Sie mit eigenen Worten, welche Maßnahmen zur Sicherheit der

Softwarequalität existieren. Kategorisieren Sie diese Maßnahmen und nennen

Sie Beispiele, wie sich diese Maßnahmen voneinander unterscheiden.

1. Präventive Maßnahmen

Ziel: Verhindern von Fehlern bereits während der Planungs- und Entwicklungsphase.

Beispiele:

- **Anforderungsanalyse:** Sorgfältige Erhebung und Dokumentation der Anforderungen, um sicherzustellen, dass das System die Bedürfnisse erfüllt.
- **Design- und Code-Reviews:** Regelmäßige Überprüfung des Designs und Codes durch Kollegen zur frühzeitigen Fehlererkennung.
- **Modularisierung und saubere Architektur:** Aufteilen der Software in klare, wartbare Module.

Unterschied: Präventive Maßnahmen setzen früh an, um Fehler zu vermeiden, bevor sie entstehen.

2. Detektive Maßnahmen

Ziel: Fehler in der Software identifizieren und analysieren.

Beispiele:

- **Testen:** Durchführung verschiedener Testarten (z. B. Unit-Tests, Integrationstests, Systemtests) zur Überprüfung der Funktionalität.
- **Code-Analyse-Tools:** Werkzeuge zur statischen und dynamischen Analyse des Codes, um potenzielle Fehler aufzudecken.
- **Fehlerprotokollierung:** Sammeln von Fehlermeldungen und Logs während des Betriebs.

Unterschied: Detektive Maßnahmen werden angewendet, um vorhandene Fehler zu finden und ihre Ursachen zu ermitteln.

3. Korrektive Maßnahmen

Ziel: Fehler beheben, sobald sie identifiziert wurden.

Beispiele:

- **Bugfixes:** Behebung von Fehlern im Code, die während des Testens oder im Betrieb gefunden wurden.
- **Refactoring:** Verbesserung des Codes ohne Änderung des äußeren Verhaltens, um Wartbarkeit und Verständlichkeit zu erhöhen.

Unterschied: Korrektive Maßnahmen setzen an, nachdem Fehler gefunden wurden, um diese zu beheben und die Software zu verbessern.

Erklären Sie in eigenen Worten, zu welchen Qualitätssicherungsmaßnahmen das Testen gehört.

1. Funktionale Tests

Überprüfen, ob die Software die gewünschten Funktionen korrekt ausführt, z. B. durch Unit-Tests und Integrationstests.

2. Nicht-funktionale Tests

Prüfen von Eigenschaften wie Leistung und Sicherheit, z. B. Leistungstests und Sicherheitstests.

3. Benutzertests

Einbeziehen der Nutzer, um die Benutzerfreundlichkeit zu bewerten, z. B. durch Usability-Tests und Akzeptanztests.

4. Regressionstests

Sicherstellen, dass Änderungen keine bestehenden Funktionen beeinträchtigen.

Aufgabe 2 „Testen und Sicherung der Softwarequalität“

Es gibt mehrere Definitionen, was Testen bedeutet. Welche Definition ist Ihrer Meinung nach die beste und WARUM?

1. Die beste Definition stammt von Glenford Myers: "Testen ist der Prozess des Ausführens eines Programms mit der Absicht, Fehler zu finden."

Warum diese Definition gut ist:

• Zielorientiert: Sie fokussiert sich darauf, aktiv Fehler zu finden, statt die Abwesenheit von Fehlern zu beweisen.

• Pragmatisch: Sie betont das Testen als Methode zur Fehlerentdeckung, nicht nur zur Bestätigung der Funktionalität.

• Proaktiv: Sie fördert eine kritische Herangehensweise, die die Softwarequalität verbessert.

2. Es gibt 7 Grundsätze des Testens. Welche sind das?

Testen zeigt Fehler, nicht deren Abwesenheit.

Exhaustives Testen ist unmöglich.

Frühes Testen spart Zeit und Geld.

Fehler konzentrieren sich oft in wenigen Bereichen.

Gleiche Tests finden irgendwann keine neuen Fehler mehr.

Testen ist kontextabhängig.

Fehlerfreiheit ist eine Illusion.

3. Erläutern Sie den Grundsatz des Testens: „Vollständiges Testen ist nicht möglich“.Argumentieren Sie Ihren Kommilitonen, warum das so ist.

Der Grundsatz „Vollständiges Testen ist nicht möglich“ bedeutet, dass es unmöglich ist, alle möglichen Eingaben und Szenarien zu testen. Das liegt an der riesigen Anzahl von Kombinationen, begrenzten Ressourcen, dynamischen Umgebungen und unvorhersehbaren Benutzeraktionen. Deshalb fokussiert man sich auf die wichtigsten Szenarien, um die wahrscheinlichsten Fehler zu finden.

4. Erläutern Sie den Grundsatz des Testens: „Trugschluss: Keine Fehler bedeutet ein brauchbares System“

Der Grundsatz „Trugschluss: Keine Fehler bedeutet ein brauchbares System“ besagt, dass selbst wenn Tests keine Fehler finden, das System nicht automatisch als brauchbar oder gut angesehen werden kann. Gründe dafür sind:

• Unvollständige Anforderungen: Das System kann zwar fehlerfrei sein, aber nicht die tatsächlichen Bedürfnisse oder Erwartungen der Benutzer erfüllen.

• Falsche Funktionalität: Die Software kann technisch korrekt arbeiten, aber die falschen Funktionen oder unpassende Lösungen anbieten.

• Usability-Probleme: Ein fehlerfreies System kann schwer zu bedienen sein, was die Benutzerakzeptanz beeinträchtigt.

Ein fehlerfreies System ist also nicht zwangsläufig ein nützliches oder zufriedenstellendes System.

Aufgabe 3 „Denkweise und Kompetenzen eines Testers“

1.Über welche 1)Kompetenzen, 2)Fähigkeiten und 3)persönliche Charaktereigenschaften sollte Ihrer Meinung nach ein erfolgreicher Tester verfügen? Ordnen Sie Ihre Ergebnisse den jeweiligen Kategorien zu (1), 2), 3)), begründen und notieren Sie sie.

Kompetenzen: Technisches Fachwissen, Programmcode verstehen, UML verstehen, Paradigmen

Fähigkeiten: Objektiv, konstruktiv, Logisches und Analytisches Denken, Fehler Kommunikation, Soziale & kommunikations Kompetenz

Charakter: penible, hartnäckig, emphatisch, neutral, faktenorientiert

2.Wann ist Ihrer Meinung nach ein Tester erfolgreich? Begründen Sie Ihre Antwort.

Ein Tester ist dann erfolgreich, wenn er möglichst viele relevante Fehler oder Schwachstellen im System identifiziert, bevor die Software in den produktiven Einsatz geht. Der Erfolg eines Testers lässt sich durch folgende Punkte begründen:

• Fehlerentdeckung: Der Tester findet nicht nur offensichtliche, sondern auch versteckte oder schwer zu reproduzierende Fehler.

• Risikominderung: Durch das Aufdecken kritischer Fehler trägt der Tester dazu bei, das Risiko von Ausfällen oder schwerwiegenden Problemen im Betrieb zu minimieren.

• Qualitätsverbesserung: Ein erfolgreicher Tester verbessert durch gründliche Tests die Qualität der Software und sorgt dafür, dass sie den Anforderungen und Erwartungen der Nutzer entspricht.

• Proaktive Fehlervermeidung: Ein Tester, der frühzeitig involviert ist, hilft, Fehler schon in der Entstehung zu vermeiden, indem er die Entwicklung durch regelmäßiges Feedback unterstützt.

• Kundenzufriedenheit: Der Tester trägt dazu bei, dass die Software stabil, benutzerfreundlich und sicher ist, was die Zufriedenheit der Endnutzer erhöht.

Aufgabe 4

1.Definieren Sie die Begriffe Fehlerwirkung, Fehlerzustand, Fehlhandlung.

Fehlerwirkung (Failure): Ein Fehler, der sich im Verhalten des Systems äußert. Es ist das sichtbare, unerwünschte Ergebnis oder die Abweichung vom erwarteten Verhalten während der Nutzung des Systems.

Fehlerzustand (Defect/Bug): Ein Defekt oder Problem im Code oder in der Logik der Software. Es ist die eigentliche Ursache, die zu einer Fehlerwirkung führt, wenn der betroffene Code ausgeführt wird.

Fehlhandlung (Error): Ein menschlicher Fehler, der während der Planung, Entwicklung oder Wartung der Software auftritt. Diese Fehlhandlung kann zu einem Fehlerzustand führen, der wiederum eine Fehlerwirkung zur Folge haben kann.

2.Was ist Fehlermaskierung?

Fehlermaskierung ist der Prozess, bei dem ein Fehler im System durch andere Funktionen oder Mechanismen verborgen wird. Dadurch bleibt der Fehler unentdeckt, was zu langfristigen Problemen führen kann.

3.Erläutern Sie den Unterschied zwischen Testen und Debugging.

Testen:

Ziel: Fehler und Probleme in der Software identifizieren.

Prozess: Systematischer Prozess mit verschiedenen Testarten (z. B. Unit-Tests).

Durchführung: Oft von Testern oder Qualitätssicherungsteams.

Ergebnis: Liefert Feedback über die Softwarequalität, ohne Ursachen zu analysieren.

Debugging:

Ziel: Ursachen von Fehlern finden und beheben.

Prozess: Iterativer Prozess, bei dem der Code analysiert wird.

Durchführung: Normalerweise von Entwicklern, die den Code geschrieben haben.

Ergebnis: Korrektur der Fehler im Code.

4. Erläutern Sie, warum jeder Test eine stichprobenartige Prüfung ist.

Jeder Test ist eine stichprobenartige Prüfung, weil es unmöglich ist, jede mögliche Kombination von Eingaben, Zuständen und Bedingungen einer Software zu testen.

5. Wozu dient eine Testbasis?

Eine Testbasis dient als Grundlage für die Planung, Entwicklung und Durchführung von Tests. Sie stellt die Informationsquelle dar, aus der die Anforderungen, Bedingungen und Kriterien für die Testfälle abgeleitet werden. Die Testbasis hilft dabei, sicherzustellen, dass die Tests auf den richtigen Anforderungen und Spezifikationen beruhen.

6. Was kann als Testbasis verwendet werden?

Als Testbasis können verschiedene Dokumente und Informationsquellen dienen, die die Anforderungen und Spezifikationen eines Systems beschreiben. Dazu gehören Anforderungsdokumente, technische Spezifikationen, Benutzerhandbücher, Systemarchitektur-Modelle sowie Lasten- und Pflichtenhefte. Auch Prozessbeschreibungen und frühere Softwareversionen können als Grundlage für Tests genutzt werden. Diese Informationen helfen dabei, Testfälle zu entwickeln, die sicherstellen, dass die Software den relevanten Anforderungen entspricht.

7. Warum soll Testen möglichst frühzeitig im Projekt beachtet und realisiert werden?

Testen sollte frühzeitig im Projekt erfolgen, weil:

Fehlerfrüherkennung: Fehler werden schneller identifiziert, was ihre Behebung einfacher macht.

Kostenreduktion: Fehlerbehebung ist in frühen Phasen kostengünstiger als später.

Risiko Minimierung: Kritische Probleme können rechtzeitig erkannt und behoben werden.

Verbesserte Qualität: Frühes Feedback fördert die kontinuierliche Verbesserung der Softwarequalität.

Bessere Planung: Frühes Testen ermöglicht eine effizientere Planung der Testaktivitäten.

8. Nennen Sie die Hauptmerkmale der Softwarequalität nach ISO 25010.

Die Hauptmerkmale der Softwarequalität nach ISO 25010 sind:

Funktionalität: Erfüllung der Anforderungen und Funktionen.

Zuverlässigkeit: Korrekte Funktion unter festgelegten Bedingungen.

Benutzerfreundlichkeit (Usability): Benutzererfahrung und Bedienbarkeit.

Effizienz: Optimale Ressourcennutzung bei geforderter Leistung.

Wartbarkeit: Leichtigkeit der Wartung und Anpassung.

Portabilität: Fähigkeit, in verschiedenen Umgebungen eingesetzt zu werden.

Sicherheit: Schutz vor unautorisiertem Zugriff und Bedrohungen.

9. Definieren Sie den Begriff Zuverlässigkeit eines Systems.

Zuverlässigkeit eines Systems bezeichnet die Fähigkeit eines Systems, unter festgelegten Bedingungen über einen bestimmten Zeitraum hinweg konsistent und korrekt zu funktionieren. Dies umfasst die folgenden Aspekte:

Stabilität: Das System sollte ohne unerwartete Ausfälle arbeiten.

Fehlertoleranz: Das System sollte in der Lage sein, Fehler zu erkennen und gegebenenfalls zu beheben, ohne dass der Benutzer beeinträchtigt wird.

Verfügbarkeit: Das System sollte jederzeit einsatzbereit sein, um die geforderten Funktionen bereitzustellen.

10. Warum sollte ein Entwickler nicht seine eigenen Programme testen?

Ein Entwickler sollte nicht seine eigenen Programme testen, weil:

Subjektivität: Der Entwickler kann voreingenommen sein und Fehler übersehen.

Kenntnis des Codes: Er kennt die Implementierungsdetails und trifft unbewusste Annahmen.

Blindheit für Fehler: Nähe zum Code kann zu blinden Flecken führen.

Unabhängige Perspektiven: Externe Tester bringen frische Sichtweisen und unterschiedliche Nutzungsszenarien ein.

Trennung der Verantwortlichkeiten: Klare Rollen verbessern die Softwarequalität.